

# Unveiling Terraform: The Infrastructure-as-Code Software Tool

Discover Terraform's power, a valuable Infrastructure as Code tool, and learn how to use it to install an nginx server.



Image Source: <https://www.nginx.com/blog/>

In the ever-evolving landscape of IT and DevOps, Infrastructure as Code (IaC) stands out as a transformative practice. It represents a fundamental shift in how organisations manage and provision their infrastructure resources, and has emerged as a key component for effective, scalable, and cost-effective infrastructure management. The primary objective of this article is to familiarise you with Terraform, an important Infrastructure as Code (IaC) tool, and walk you through the procedure of deploying an nginx server utilising its features.

After reading this article, you will have a basic understanding of Terraform and will be able to use it to install an nginx server. You will understand the importance of Infrastructure as Code in simplifying and automating the deployment process, paving the path for effective management of more

complex infrastructure setups. So, whether you're an upcoming DevOps engineer, an experienced developer, or an infrastructure enthusiast, this article will help you understand Terraform's strength and potential for orchestrating Infrastructure as Code.

## Introducing Terraform

HashiCorp's Terraform is a business-source Infrastructure as Code (IaC) tool. It helps organisations declaratively create, provide, and manage their infrastructure and services, treating infrastructure configurations as code. Terraform is designed to interact smoothly with a variety of cloud providers, on-premises systems, and third-party services, making it a versatile and extensively used solution in the cloud computing and DevOps fields. Some of the key features of Terraform are as follows.

### ***Declarative configuration:***

Terraform describes infrastructure resources and their desired state using a declarative language (HashiCorp Configuration Language or HCL). Users describe 'what' they want rather than 'how' to get it.

### ***Multi-cloud and multi-provider***

**support:** Its large provider ecosystem supports various cloud providers (AWS, Azure, Google Cloud, and so on) and third-party services. From a single setup, users may manage resources across numerous platforms.

**Resource management:** Terraform's resource architecture enables users to define and manage infrastructure components such as virtual machines, networks, databases, and more.

**State management:** It keeps track of the current state of controlled infrastructure in a state file. This state file assists Terraform in determining